**Question 1:** What is a good choice for a primary key here? In contrast, give an example of an attribute (or composite) that would not be a valid primary key.

Good Choice for Primary Key: **fund_id**

The best choice for a primary key in this dataset is **fund_id**.

Reasons why fund_id works as a primary key:

- Every row has a unique fund_id value (ranging from 1 to 523)
- No fund_id is repeated or NULL
- Each fund_id represents exactly one funding event, which is what each row in the table describes
- I verified this by sorting the data in Excel by fund_id column and confirmed there are no duplicate values

Invalid Primary Key Example: company

The company column would **NOT** work as a valid primary key.

Reasons why company fails as a primary key:

- Many companies appear multiple times in the dataset because they received funding in different rounds
- For example, Facebook appears 7 times (rows 8-14) for different funding events
- A primary key must uniquely identify each row, but company alone cannot do this because the same company can have many funding events
- Even combining company + round wouldn't work because some companies have multiple funding events in the same round (Facebook has three separate Series C rounds in rows 11, 12, and 13)

**Question 2:** Does the table satisfy First Normal Form (1NF)?

Answer: **YES,** the table satisfies 1NF

Explanation: First Normal Form requires:

1. All values must be atomic (meaning each cell contains only one single value, not lists or multiple values)
2. Each column must contain values of the same data type
3. Each column must have a unique name

4. No repeating groups of columns

Why this table satisfies 1NF:

- Every cell in the table contains a single atomic value. For example, the company column has one company name per cell, raisedAmt has one amount per cell, etc.
- There are no cells with comma-separated values or lists
- Each column has one data type: fund_id contains numbers, company contains text, fundedDate contains dates, etc.
- All columns have unique names (fund_id, company, numEmps, category, city, state, fundedDate, raisedAmt, raisedCurrency, round)
- There are no repeating column groups like "investor1, investor2, investor3"

## **Question 3:** Does the table satisfy Second Normal Form (2NF)?

Answer: **YES**, the table satisfies 2NF (but has other normalization issues)

Explanation: Second Normal Form requires:

1. The table must be in 1NF (which we confirmed above)
2. All non-key attributes must be fully functionally dependent on the entire primary key (no partial dependencies)

Why this table satisfies 2NF:

- Our primary key is fund_id, which is a single column (not a composite key made of multiple columns)
- When the primary key is just one column, partial dependencies cannot exist because there's no "part" of the key for attributes to depend on
- All attributes in each row (company, numEmps, category, city, state, fundedDate, raisedAmt, raisedCurrency, round) are related to that specific funding event identified by fund_id
- Therefore, the table technically satisfies 2NF

However, there is a practical problem:

Some attributes like numEmps, category, city, and state describe the company itself, not the specific funding event. For example, Facebook appears in 7 rows with numEmps = 450 in all of them, even though these funding events happened over multiple years (2004-2008), and employee count definitely changed during this time. This causes data redundancy and potential inconsistencies, which will be addressed when we check 3NF

## **Question 4:** Does the table satisfy Third Normal Form (3NF)?

Answer: **NO**, the table does NOT satisfy 3NF

Explanation: Third Normal Form requires:

1. The table must be in 2NF (which we confirmed above)
2. There must be no transitive dependencies (meaning non-key attributes should not depend on other non-key attributes)

Why this table fails 3NF - Multiple violations:

Violation 1: State depends on City (Transitive Dependency**)**

- The state column depends on the city column, not directly on fund_id
- If you know the city is "San Francisco," you automatically know the state is "CA"
- This means: fund_id → city → state (state is transitively dependent on fund_id through city)
- Example: Rows 6, 7, 19, 20, 21, 22, 23, etc. all have city = "San Francisco" and state = "CA"

Violation 2: Company-related attributes create transitive dependencies

- Attributes like company, numEmps, and category describe the company, not the specific funding event
- The dependency chain is: fund_id → company → numEmps, category, city, state
- These attributes are transitively dependent on fund_id through the company
- Example: Look at Digg (rows 6-7), Facebook (rows 8-14), or Plaxo (rows 24-27) - the same company information is repeated for every funding event

Violation 3: Data Redundancy Issues

- Company information is repeated multiple times, causing redundancy
- For example, "Facebook, 450 employees, web category, Palo Alto, CA" is stored 7 times (rows 8-14)
- If Facebook's category or location changes, we would need to update multiple rows, risking inconsistency
- This redundancy violates the goal of 3NF, which is to eliminate these types of dependencies

Specific Examples from the Data:

- Facebook (rows 8-14): Same company info repeated 7 times
- Digg (rows 6-7): Company info repeated 2 times
- Plaxo (rows 24-27): Company info repeated 4 times

To achieve 3NF, we need to separate this data into multiple related tables to eliminate these transitive dependencies.

## **Question 5:** Entity-Relationship Diagram to Achieve 3NF

Proposed Database Design: To bring this dataset into Third Normal Form, we need to create **four separate tables** to eliminate redundancy and transitive dependencies:

**Table 1: COMPANIES**

COMPANIES

---

company_id (PK)

company_name

category

- Purpose: Stores unique company information
- Primary Key: company_id
- Example rows:
  - (1, 'Facebook', 'web')
  - (2, 'Digg', 'web')

**Table 2: LOCATIONS**

LOCATIONS

---

location_id (PK)

city

state

- Purpose: Stores unique city-state combinations
- Primary Key: location_id
- Example rows:

    - (1, 'Palo Alto', 'CA')
    - (2, 'San Francisco', 'CA')
    - (3, 'Boston', 'MA')

**Table 3: COMPANY_DETAILS**

COMPANY_DETAILS

---

company_id (PK, FK)

location_id (FK)

numEmps

- Purpose: Links companies to their locations and stores employee count
- Primary Key: company_id
- Foreign Keys:
    - company_id references COMPANIES(company_id)
    - location_id references LOCATIONS(location_id)
- Example rows:
    - (1, 1, 450) -- Facebook, Palo Alto, 450 employees
    - (2, 2, 60) -- Digg, San Francisco, 60 employees

**Table 4: FUNDING_EVENTS**

FUNDING_EVENTS

---

fund_id (PK)

company_id (FK)

fundedDate

raisedAmt

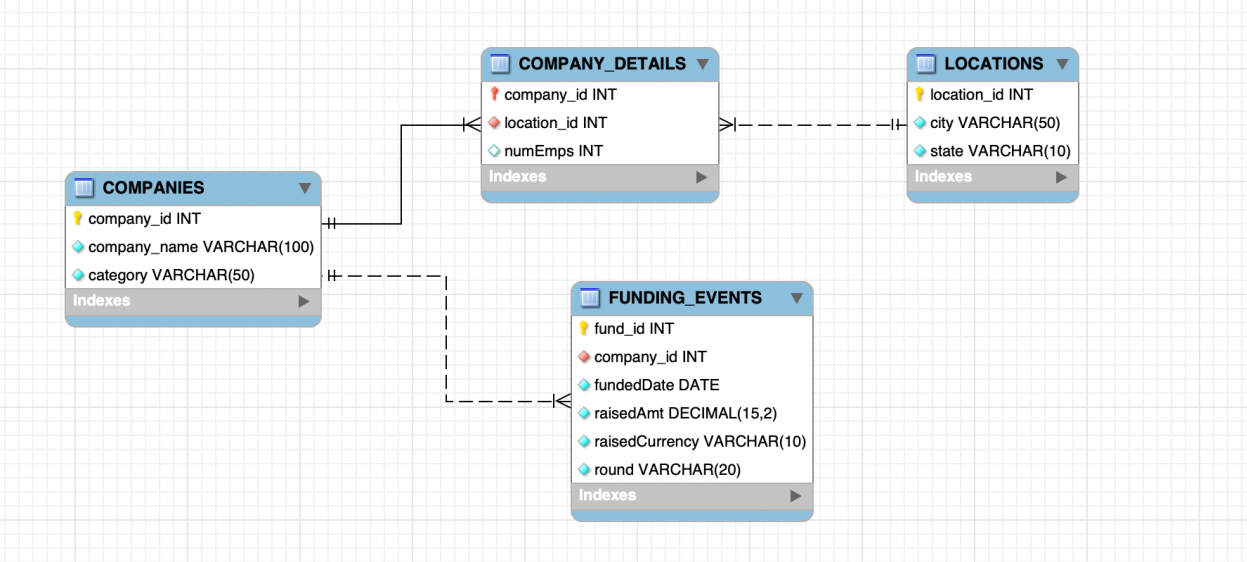
raisedCurrency

round

- Purpose: Stores individual funding events
- Primary Key: fund_id
- Foreign Key: company_id references COMPANIES(company_id)
- Example rows:

- (8, 1, '9/1/2004', 500000, 'USD', 'angel') -- Facebook angel round
- (9, 1, '5/1/2005', 12700000, 'USD', 'a') -- Facebook Series A

## Entity-Relationship Diagram:




### Relationships Explained:

1. COMPANIES to FUNDING_EVENTS: One-to-Many (1:M)
   - One company can have many funding events
   - Each funding event belongs to exactly one company
   - Example: Facebook (1 company) has 7 funding events
2. COMPANIES to COMPANY_DETAILS: One-to-One (1:1)
   - Each company has exactly one set of company details (location and employee count)
   - Each company detail record belongs to exactly one company
3. LOCATIONS to COMPANY_DETAILS: One-to-Many (1:M)
   - One location can be home to many companies
   - Each company is located in exactly one location
   - Example: "Palo Alto, CA" is the location for multiple companies like Facebook, Ning, Box.net, etc.

### Benefits of This Design:

Eliminates Redundancy: Company information is stored only once, not repeated for every funding event. Removes Transitive Dependencies: State no longer depends on city in the same table; they're in a separate LOCATIONS table. Achieves 3NF: All non-key attributes depend only on the primary key in their respective tables. Easier Updates: If a company changes location

or category, we update only one row instead of multiple rows. Data Integrity: Reduces chances of inconsistent data (like having different employee counts for the same company)

## PART 2: RECIPE DATABASE CASE STUDY

**Question 1:** Insert New Recipes

I successfully inserted two new Indian recipes into the recipe database:

Recipe 1: Masala Chai (Indian Spiced Tea)

- Category: Beverage

- Prep Time: 5 minutes

- Cook Time: 10 minutes

- Servings: 4

- Difficulty: 1 (Easy)

- Ingredients: 9 ingredients including water, black tea leaves, whole milk,  white sugar, fresh ginger, cardamom pods, cinnamon stick, black peppercorns, and whole cloves

Recipe 2: Aloo Paratha (Potato Stuffed Flatbread)

- Category: Bread

- Prep Time: 20 minutes

- Cook Time: 20 minutes

- Servings: 4

- Difficulty: 2 (Medium)

- Ingredients: 10 ingredients including whole wheat flour, boiled potatoes, chopped onion, green chili, fresh coriander leaves, cumin seeds, garam masala, salt, vegetable oil, and butter

The following data was inserted:

- 2 new categories (Beverage, Bread)

- 2 new recipes (Masala Chai, Aloo Paratha)

- 16 new ingredients (various Indian spices and cooking ingredients)

- 19 recipe-ingredient relationships (9 for Masala Chai, 10 for Aloo Paratha)

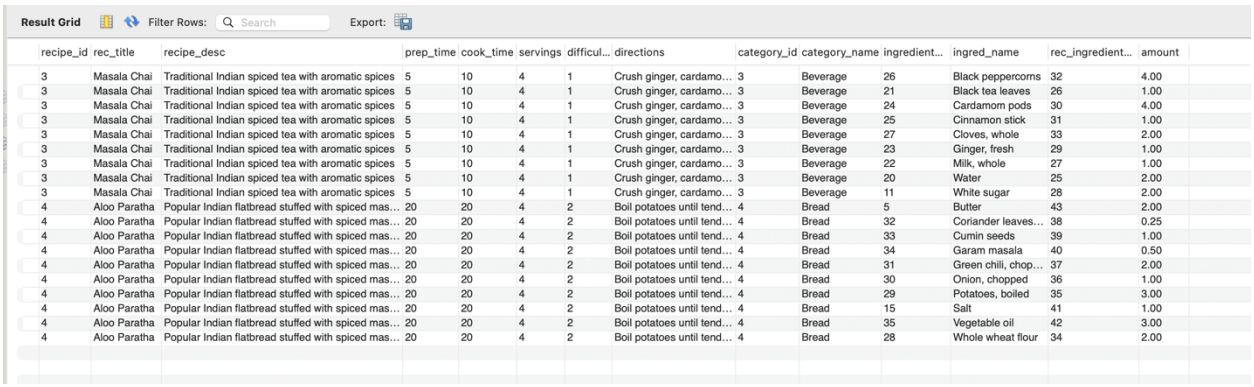
**Question 2:** SQL Query Returning All Information

This query uses JOIN statements to combine data from all four tables (recipe_main, categories, ingredients, and rec_ingredients) and returns complete information for only the two new recipes I inserted.

**SQL Query**:

```
SELECT
    rm.recipe_id,
    rm.rec_title,
    rm.recipe_desc,
    rm.prep_time,
    rm.cook_time,
    rm.servings,
    rm.difficulty,
    rm.directions,
    c.category_id,
    c.category_name,
    i.ingredient_id,
    i.ingred_name,
    ri.rec_ingredient_id,
    ri.amount
FROM recipe_main rm
JOIN categories c ON rm.category_id = c.category_id
JOIN rec_ingredients ri ON rm.recipe_id = ri.recipe_id
JOIN ingredients i ON ri.ingredient_id = i.ingredient_id
WHERE rm.recipe_id IN (3, 4)
ORDER BY rm.recipe_id, i.ingred_name;
```

**Query Results**:



Result Grid | Filter Rows: Search | Export:

| recipe_id | rec_title | recipe_desc | prep_time | cook_time | servings | difficul... | directions | category_id | category_name | ingredient... | ingred_name | rec_ingredient... | amount |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 26 | Black peppercorns | 32 | 4.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 21 | Black tea leaves | 26 | 1.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 24 | Cardamom pods | 30 | 4.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 25 | Cinnamon stick | 31 | 1.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 27 | Cloves, whole | 33 | 2.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 23 | Ginger, fresh | 29 | 1.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 22 | Milk, whole | 27 | 1.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 20 | Water | 25 | 2.00 |
| 3 | Masala Chai | Traditional Indian spiced tea with aromatic spices | 5 | 10 | 4 | 1 | Crush ginger, cardamo... | 3 | Beverage | 11 | White sugar | 28 | 2.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 5 | Butter | 43 | 2.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 32 | Coriander leaves... | 38 | 0.25 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 33 | Cumin seeds | 39 | 1.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 34 | Garam masala | 40 | 0.50 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 31 | Green chili, chop... | 37 | 2.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 30 | Onion, chopped | 36 | 1.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 29 | Potatoes, boiled | 35 | 3.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 15 | Salt | 41 | 1.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 35 | Vegetable oil | 42 | 3.00 |
| 4 | Aloo Paratha | Popular Indian flatbread stuffed with spiced mas... | 20 | 20 | 4 | 2 | Boil potatoes until tend... | 4 | Bread | 28 | Whole wheat flour | 34 | 2.00 |

The query successfully returned all rows representing all ingredient combinations for both recipes, with complete information from all four tables as required.

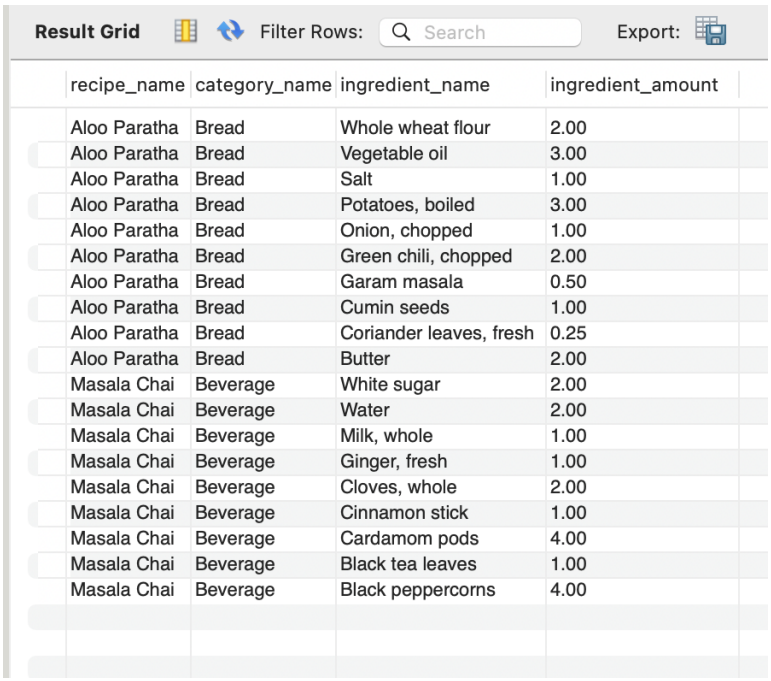
**Question 3**: Sorted SELECT Query

This query returns only the recipe name, category name, ingredient name, and ingredient amount, sorted by descending category name, then ascending recipe name, then descending ingredient name.

**SQL Query**:

```
SELECT
    rm.rec_title AS recipe_name,
    c.category_name,
    i.ingred_name AS ingredient_name,
    ri.amount AS ingredient_amount
FROM recipe_main rm
JOIN categories c ON rm.category_id = c.category_id
JOIN rec_ingredients ri ON rm.recipe_id = ri.recipe_id
JOIN ingredients i ON ri.ingredient_id = i.ingredient_id
WHERE rm.recipe_id IN (3, 4)
ORDER BY c.category_name DESC, rm.rec_title ASC, i.ingred_name DESC;
```

**Query Results**:




| recipe_name | category_name | ingredient_name | ingredient_amount |
|---|---|---|---|
| Aloo Paratha | Bread | Whole wheat flour | 2.00 |
| Aloo Paratha | Bread | Vegetable oil | 3.00 |
| Aloo Paratha | Bread | Salt | 1.00 |
| Aloo Paratha | Bread | Potatoes, boiled | 3.00 |
| Aloo Paratha | Bread | Onion, chopped | 1.00 |
| Aloo Paratha | Bread | Green chili, chopped | 2.00 |
| Aloo Paratha | Bread | Garam masala | 0.50 |
| Aloo Paratha | Bread | Cumin seeds | 1.00 |
| Aloo Paratha | Bread | Coriander leaves, fresh | 0.25 |
| Aloo Paratha | Bread | Butter | 2.00 |
| Masala Chai | Beverage | White sugar | 2.00 |
| Masala Chai | Beverage | Water | 2.00 |
| Masala Chai | Beverage | Milk, whole | 1.00 |
| Masala Chai | Beverage | Ginger, fresh | 1.00 |
| Masala Chai | Beverage | Cloves, whole | 2.00 |
| Masala Chai | Beverage | Cinnamon stick | 1.00 |
| Masala Chai | Beverage | Cardamom pods | 4.00 |
| Masala Chai | Beverage | Black tea leaves | 1.00 |
| Masala Chai | Beverage | Black peppercorns | 4.00 |

The results show proper sorting with Bread category appearing first (descending), followed by Beverage category. Within each category, ingredients are sorted in descending alphabetical order as required.